



WebGL

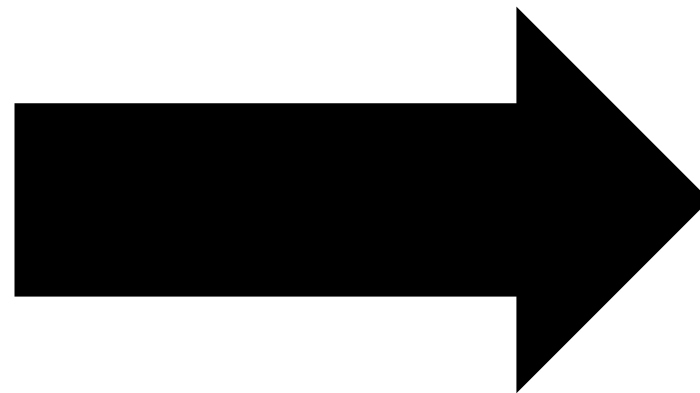
COMPUTAÇÃO GRÁFICA

Alunos:

- Fabio Azevedo
- Pedro Mikhael
- Rian Vilanova
- João Victor
- Bianca Leão

Sistema de Visualização

introdução



Em Computação Gráfica, mesmo quando se trabalha com cenas e jogos tridimensionais, o objetivo final do sistema de visualização é produzir uma imagem bidimensional que possa ser exibida em um dispositivo de saída, como o monitor. Isso acontece porque a tela é inerentemente 2D, composta por pixels organizados em linhas e colunas. Assim, todo o pipeline gráfico existe para transformar uma descrição geométrica tridimensional do mundo em uma imagem plana que preserve a sensação de profundidade, orientação e perspectiva.

Sistema de Visualização

introdução

- Sistema de Coordenadas do Mundo (WCS)

O processo inicia-se no Sistema de Coordenadas do Mundo (World Coordinate System - WCS), que define um referencial global para toda a cena.

- Definição dos objetos da cena

Nesse sistema, todos os objetos são descritos por coordenadas tridimensionais (x,y,z) (x, y, z) (x,y,z) .

- Vértices e transformações

Cada vértice de um objeto, assim como suas transformações de posição, rotação e escala, é especificado em relação a esse referencial global.

- Independência da câmera

As definições no WCS são independentes da posição ou orientação da câmera.

- Ausência de visualização

Nesse estágio, não há ainda noção de visibilidade ou de como a cena será observada; existe apenas a descrição geométrica dos objetos no espaço tridimensional.

Sistema de Visualização

Câmera



Para que a cena possa ser observada, é necessário definir uma câmera virtual, que representa o ponto de vista do observador. A câmera também é definida em coordenadas do mundo, por meio de três parâmetros principais: a posição da câmera, chamada de EYE ou VRP (View Reference Point); a direção para onde a câmera aponta, representada pelo vetor VPN (View Plane Normal); e o vetor UP (ou VUV), que indica qual direção é considerada “para cima” pela câmera. Esses parâmetros permitem construir um novo sistema de coordenadas, associado à câmera, chamado de Sistema de Coordenadas da Câmera ou Viewing Reference Coordinates (VRC).

Sistema de Visualização

Transformação

- Objetivo da transformação de visualização

A transformação de visualização tem como objetivo mudar o referencial dos pontos do mundo (WCS) para o referencial da câmera (VRC). Isso equivale a reescrever todos os pontos da cena como se a câmera estivesse posicionada na origem $(0,0,0)$ e orientada de forma padrão, normalmente apontando para o eixo $-z$. Esse procedimento simplifica as etapas seguintes do pipeline gráfico.

- Construção da base ortonormal da câmera

Para realizar essa transformação, constrói-se inicialmente uma base ortonormal associada à câmera, formada pelos vetores u , v e n .

- Vetor de visão n

O vetor n representa a direção de visão da câmera e é obtido pela normalização do vetor VPN (View Plane Normal):

$$\vec{n} = \frac{\text{VPN}}{\|\text{VPN}\|}$$

Sistema de Visualização

Transformação

- Correção do vetor UP

O vetor UP fornecido pelo usuário deve ser ajustado para garantir que seja ortogonal ao vetor n . Para isso, remove-se a componente de UP na direção de n , obtendo o vetor corrigido:

$$UP' = UP - n(n \cdot UP)$$

- Vetor vertical v

O vetor corrigido UP' é então normalizado, formando o vetor v , que representa o eixo vertical da câmera:

$$v = \frac{UP'}{\|UP'\|}$$

- Vetor horizontal u

O vetor u , que define o eixo horizontal da câmera, é obtido por meio do produto vetorial entre os vetores n e v :

$$u = n \times v$$

- Sistema de coordenadas da câmera

Os vetores u , v e n formam um sistema de coordenadas ortonormal associado à câmera.

Sistema de Visualização

Transformação

- Matriz de rotação

A matriz de rotação é responsável por alinhar os eixos do mundo aos eixos da câmera. Ela é definida como:

$$M_{rot} = \begin{bmatrix} u_x & u_y & u_z & 0 \\ v_x & v_y & v_z & 0 \\ -n_x & -n_y & -n_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

- Matriz de translação

Além da rotação, é necessário transladar a cena de forma que a câmera fique na origem. Isso é feito deslocando todos os pontos do mundo pelo vetor -EYE:

$$M_{trans} = \begin{bmatrix} 1 & 0 & 0 & -EY E_x \\ 0 & 1 & 0 & -EY E_y \\ 0 & 0 & 1 & -EY E_z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Sistema de Visualização

Transformação

- Matriz de visualização

A matriz de visualização é obtida pela composição da rotação com a translação:

$$M_{vis} = M_{rot} \cdot M_{trans}$$

- Forma clássica da matriz de visualização

Expandindo essa multiplicação, obtém-se a forma clássica da matriz de visualização:

$$M_{vis} = \begin{bmatrix} u_x & u_y & u_z & -u \cdot EYE \\ v_x & v_y & v_z & -v \cdot EYE \\ n_x & n_y & n_z & -n \cdot EYE \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Sistema de Visualização

Código

```
const MAZE_MIN_X = -1.1;  
const MAZE_MAX_X = 1.0;  
const MAZE_MIN_Z = -1.1;  
const MAZE_MAX_Z = 1.0;
```

```
let cameraX = 0;  
let cameraY = 0.05;  
let cameraZ = 0;
```

```
let cameraYaw = 0;  
let cameraPitch = 0;
```

```
// 2. CONFIGURAÇÃO DA CÂMERA (VIEW MATRIX)  
const eye = [cameraX, cameraY, cameraZ];  
const lookX = cameraX + Math.sin(cameraYaw) * Math.cos(cameraPitch);  
const lookY = cameraY + Math.sin(cameraPitch);  
const lookZ = cameraZ - Math.cos(cameraYaw) * Math.cos(cameraPitch);  
const center = [lookX, lookY, lookZ];  
const up = [0, 1, 0];  
const viewMatrix = lookAt(eye, center, up);  
  
const aspect = gl.canvas.width / gl.canvas.height;  
const projectionMatrix = perspective(Math.PI / 4, aspect, 0.1, 100.0);
```

Sistema de Visualização

Código

```
function updateCamera() {  
    // Calcula direção para frente baseada no y  
    const forwardX = Math.sin(cameraYaw);  
    const forwardZ = -Math.cos(cameraYaw);  
  
    // Direção para os lados (perpendicular)  
    const rightX = Math.cos(cameraYaw);  
    const rightZ = Math.sin(cameraYaw);  
  
    // Movimento WASD (apenas no plano XZ)  
    if (keys['w']) {  
        cameraX += forwardX * moveSpeed;  
        cameraZ += forwardZ * moveSpeed;  
    }  
    if (keys['s']) {  
        cameraX -= forwardX * moveSpeed;  
        cameraZ -= forwardZ * moveSpeed;  
    }  
    if (keys['a']) {  
        cameraX -= rightX * moveSpeed;  
        cameraZ -= rightZ * moveSpeed;  
    }  
    if (keys['d']) {  
        cameraX += rightX * moveSpeed;  
        cameraZ += rightZ * moveSpeed;  
    }  
}
```

Textura

Código

O processo de aplicação de texturas em WebGL inicia-se com o carregamento das imagens na memória da CPU, antes que elas possam ser utilizadas pela GPU. No código, esse papel é desempenhado pelo array `texImages` e pelo contador `loadedTexCount`, que controlam o carregamento assíncrono das imagens:

```
var texImages = [];  
var loadedTexCount = 0;
```

A função `loadTexturesFromSources` recebe um conjunto de URLs e cria um objeto `Image` para cada uma delas. Cada imagem é carregada de forma assíncrona e, somente quando todas estiverem completamente carregadas, o callback é acionado. Nesse estágio, as imagens ainda não são texturas WebGL; elas existem apenas como dados na memória principal:

```
function loadTexturesFromSources(texSources, onAllLoaded) {  
    loadedTexCount = 0;  
    texImages = [];  
  
    for (var i = 0; i < texSources.length; i++) {  
        texImages[i] = new Image();  
        texImages[i].src = texSources[i];  
        texImages[i].onload = function () {  
            loadedTexCount++;  
            console.log("Textura carregada: " + loadedTexCount + "/" + texSources.length);  
            if (loadedTexCount == texSources.length) {  
                onAllLoaded(texImages);  
            }  
        };  
        texImages[i].onerror = function () {  
            console.error("Erro ao carregar textura: " + this.src);  
        };  
    }  
}
```

Textura

Código

Após o carregamento da imagem, o próximo passo é enviá-la para a GPU e transformá-la em uma textura propriamente dita. Isso é feito pela função `createTextureFromImage`. Inicialmente, cria-se um objeto de textura na GPU e define-se a unidade de textura ativa, permitindo que múltiplas texturas sejam usadas simultaneamente pelos shaders:

```
var tex = gl.createTexture();  
gl.activeTexture(gl.TEXTURE0 + textureUnit);  
gl.bindTexture(gl.TEXTURE_2D, tex);
```

Em seguida, são configurados os parâmetros que determinam o comportamento da textura. Os parâmetros `TEXTURE_WRAP_S` e `TEXTURE_WRAP_T` definem que a textura deve se repetir quando as coordenadas de textura ultrapassarem o intervalo $[0,1][0,1][0,1]$, o que é especialmente útil para superfícies como pisos e paredes. Já os filtros de minificação e magnificação controlam como a textura é suavizada quando o objeto está distante ou próximo da câmera:

```
gl.texParameteri(gl.TEXTURE_2D, gl.TEXTURE_WRAP_S, gl.REPEAT);  
gl.texParameteri(gl.TEXTURE_2D, gl.TEXTURE_WRAP_T, gl.REPEAT);  
gl.texParameteri(gl.TEXTURE_2D, gl.TEXTURE_MIN_FILTER, gl.LINEAR_MIPMAP_LINEAR);  
gl.texParameteri(gl.TEXTURE_2D, gl.TEXTURE_MAG_FILTER, gl.LINEAR);
```


Textura

Código

Com os parâmetros definidos, a imagem é finalmente transferida da memória da CPU para a memória da GPU por meio da função `texImage2D`. A partir desse momento, a imagem passa a existir como uma textura acessível pelos shaders:

```
gl.texImage2D(gl.TEXTURE_2D, 0, gl.RGBA, gl.RGBA, gl.UNSIGNED_BYTE, image);
```

Logo após o envio da imagem, são gerados os mipmap da textura. Os mipmap são versões reduzidas da imagem original, utilizadas automaticamente pela GPU para melhorar a qualidade visual e o desempenho quando a textura é visualizada a diferentes distâncias:

```
gl.generateMipmap(gl.TEXTURE_2D);
```

Em algumas situações, como no caso de paredes extensas, é necessário garantir explicitamente que a textura se repita. A função `setTextureRepeat` reforça essa configuração, assegurando o comportamento de tiling da textura:

```
function setTextureRepeat(gl, texture) {  
    gl.bindTexture(gl.TEXTURE_2D, texture);  
    gl.texParameteri(gl.TEXTURE_2D, gl.TEXTURE_WRAP_S, gl.REPEAT);  
    gl.texParameteri(gl.TEXTURE_2D, gl.TEXTURE_WRAP_T, gl.REPEAT);  
}
```

Textura

Código

O código também apresenta uma versão moderna do carregamento de texturas utilizando `async/await`, encapsulada na função `carregarTextura`. Nela, o carregamento da imagem é tratado como uma `Promise`, o que simplifica o controle do fluxo de execução e torna o código mais legível:

```
const image = new Image();
image.crossOrigin = "anonymous";

image.onload = () => {
  const texture = createTextureFromImage(gl, image, 0);
  setTextureRepeat(gl, texture);
  console.log(`Textura carregada: ${url} (${image.width}x${image.height})`);
  resolve(texture);
};
```

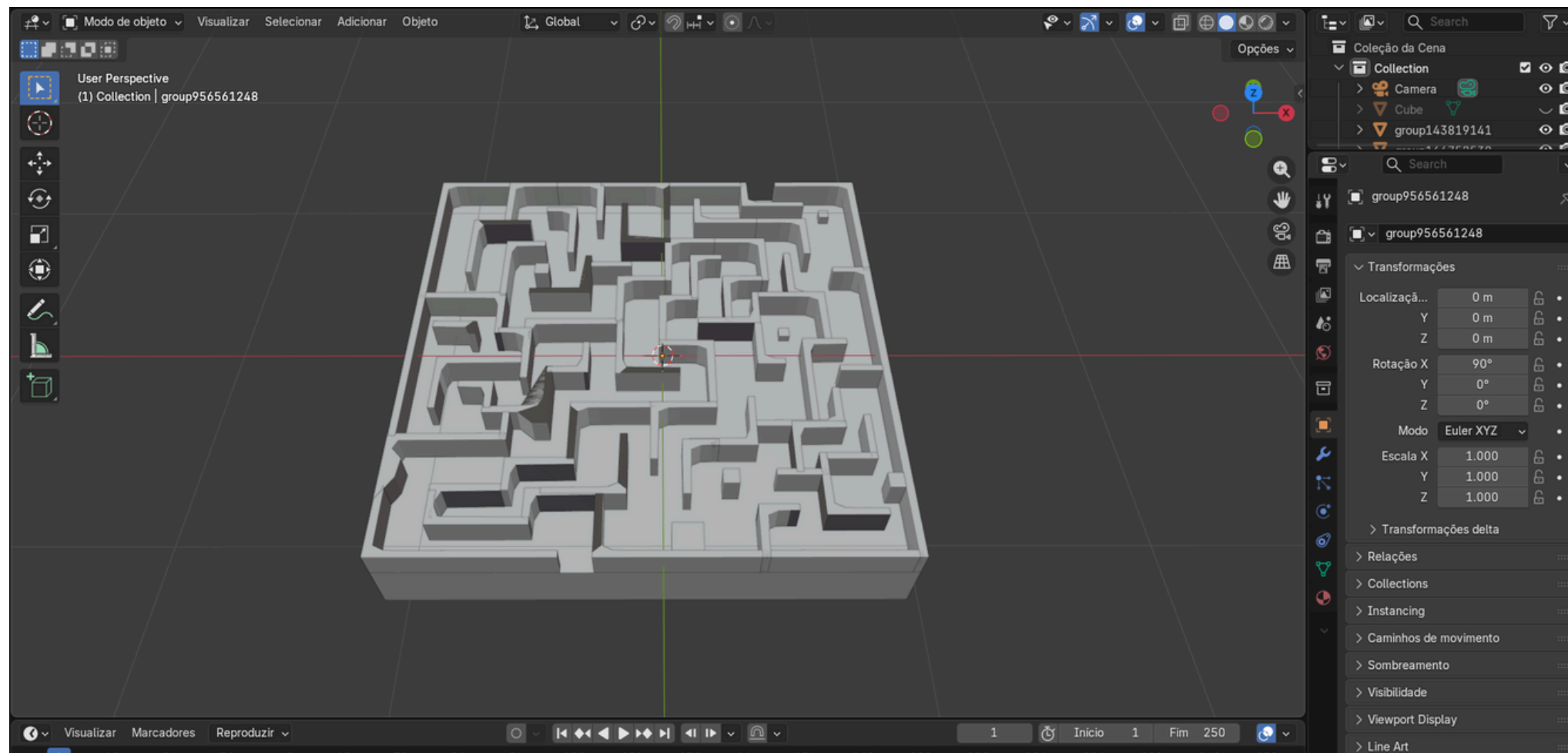
Textura

UV mapping

Uma vez criada, a textura é utilizada no pipeline de renderização por meio dos shaders. Cada vértice do objeto possui coordenadas de textura (UV), que são interpoladas ao longo do polígono. No fragment shader, essas coordenadas são usadas para consultar a textura e obter a cor correta de cada pixel, resultando no efeito visual de uma imagem aplicada sobre a geometria.

Dessa forma, o sistema de texturas em WebGL realiza uma sequência bem definida de etapas: carregamento da imagem na CPU, criação da textura na GPU, configuração de parâmetros, geração de mipmaps e, por fim, amostragem da textura durante a rasterização. Esse processo permite enriquecer visualmente os objetos da cena, mantendo eficiência e realismo gráfico.

Modelagem e Preparação dos objetos (Blender)



- Centralização e Escala na Origem: Os modelos são criados/importados e centralizados no ponto (0,0,0) do Blender. Isso facilita a aplicação de matrizes de translação e escala.
- Geometria Triangulada: Como o WebGL trabalha nativamente com triângulos, realizamos a conversão de faces quadrangulares em triângulos durante a exportação.

Leitor obj

O carregamento de modelos tridimensionais no formato OBJ consiste na conversão de uma descrição textual de geometria em buffers utilizáveis pelo WebGL. O processo inicia-se com a leitura assíncrona do arquivo .obj por meio da API fetch, obtendo seu conteúdo em forma de texto:

```
async function carregarOBJ(url) {  
  const response = await fetch(url);  
  const text = await response.text();  
  return parseOBJ(text);  
}
```

Esse conteúdo é então interpretado pelo parser, que identifica vértices (v), normais (vn), coordenadas de textura (vt) e faces (f). As faces são automaticamente trianguladas, uma vez que o WebGL trabalha exclusivamente com triângulos:

```
for (let i = 1; i < faceVertices.length - 1; i++) {  
  // Triangula: 0, i, i+1  
  processarVertice(faceVertices[0]);  
  processarVertice(faceVertices[i]);  
  processarVertice(faceVertices[i + 1]);  
}  
break;
```

Leitor obj

```
function normalizarModelo(modelo) {  
    const numVertices = vertices.length / 3;  
  
    let minX = Infinity, minY = Infinity, minZ = Infinity;  
    let maxX = -Infinity, maxY = -Infinity, maxZ = -Infinity;  
  
    for (let i = 0; i < numVertices; i++) {  
        minX = Math.min(minX, vertices[i * 3]);  
        minY = Math.min(minY, vertices[i * 3 + 1]);  
        minZ = Math.min(minZ, vertices[i * 3 + 2]);  
        maxX = Math.max(maxX, vertices[i * 3]);  
        maxY = Math.max(maxY, vertices[i * 3 + 1]);  
        maxZ = Math.max(maxZ, vertices[i * 3 + 2]);  
    }  
  
    const centerX = (minX + maxX) / 2;  
    const centerY = (minY + maxY) / 2;  
    const centerZ = (minZ + maxZ) / 2;  
    const maxSize = Math.max(maxX - minX, maxY - minY, maxZ - minZ);  
    const scaleFactor = maxSize > 0 ? 1.0 / maxSize : 1.0;  
  
    const normalizedVertices = new Float32Array(vertices.length);  
    for (let i = 0; i < numVertices; i++) {  
        normalizedVertices[i * 3] = (vertices[i * 3] - centerX) * scaleFactor;  
        normalizedVertices[i * 3 + 1] = (vertices[i * 3 + 1] - centerY) * scaleFactor;  
        normalizedVertices[i * 3 + 2] = (vertices[i * 3 + 2] - centerZ) * scaleFactor;  
    }  
  
    return {  
        vertices: normalizedVertices,  
        normais: modelo.normais,  
        texturas: modelo.texturas,  
        cores: modelo.cores,  
        indices: modelo.indices,  
        numVertices: modelo.numVertices,  
        numFaces: modelo.numFaces  
    };  
}
```

Normalização Automática: Função que recalcula as coordenadas para centralizar o objeto na origem e escalá-lo para um cubo unitário, tornando o modelo independente de sua escala original no Blender.

O código percorre todos os vértices do modelo para encontrar os limites espaciais (Bounding Box) através dos valores mínimos e máximos de X, Y e Z. A partir desses limites, calculamos o centro geométrico e o fator de escala necessário para comprimir o objeto em um cubo unitário

Leitor .mtl

```
case 'Ka':  
    // Cor ambiente  
    if (currentMaterial) {  
        materials[currentMaterial].ambient = [  
            parseFloat(partes[1]),  
            parseFloat(partes[2]),  
            parseFloat(partes[3])  
        ];  
    }  
    break;  
  
case 'Kd':  
    // Cor difusa  
    if (currentMaterial) {  
        materials[currentMaterial].diffuse = [  
            parseFloat(partes[1]),  
            parseFloat(partes[2]),  
            parseFloat(partes[3])  
        ];  
    }  
    break;  
  
case 'Ks':  
    // Cor especular  
    if (currentMaterial) {  
        materials[currentMaterial].specular = [  
            parseFloat(partes[1]),  
            parseFloat(partes[2]),  
            parseFloat(partes[3])  
        ];  
    }  
    break;
```

```
switch (comando) {  
    case 'newmtl':  
        // Novo material  
        currentMaterial = partes[1];  
        materials[currentMaterial] = {  
            name: currentMaterial,  
            // Cores padrão  
            ambient: [0.1, 0.1, 0.1], // Ka  
            diffuse: [0.8, 0.8, 0.8], // Kd  
            specular: [1.0, 1.0, 1.0], // Ks  
            emissive: [0.0, 0.0, 0.0], // Ke  
            shininess: 32.0, // Ns  
            opacity: 1.0, // d ou Tr  
            illum: 2, // Modelo de iluminação  
            // Texturas  
            diffuseMap: null, // map_Kd  
            specularMap: null, // map_Ks  
            bumpMap: null, // map_Bump ou bump  
            normalMap: null, // map_Kn  
        };  
        break;
```

Propriedades de Iluminação: Extração das constantes de Phong: Ka (Ambiente), Kd (Difusa) e Ks (Especular).

Integração com a main

A função `prepararModelo` atua como o orquestrador do pipeline gráfico, sendo responsável por centralizar o carregamento assíncrono dos modelos OBJ e seus respectivos arquivos de materiais MTL. Após a extração dos dados, ela aplica a função `normalizarModelo` em todos os objetos, o que garante a independência de escala original e facilita as transformações subsequentes de escala, rotação e translação no espaço tridimensional.

```
try {
  async function prepararModelo(path, normalizar = true) {
    const rawData = await carregarOBJComMTL(path);
    if (!rawData.vertices || rawData.vertices.length === 0) return null;
    let finalData = normalizar ? normalizarModelo(rawData) : rawData;
    if (!finalData.cores && rawData.cores) finalData.cores = rawData.cores;
    return criarBuffersOBJComCores(gl, finalData);
  }

  mazeBuffers = await prepararModelo('modelos/Maze.0/model.obj', false);
}
```

A utilização da constante `renderM` permite a que os objetos sejam carregados no mapa, posicionando os objetos carregados no mundo 3D através de transformações de matrizes.

```
// 3. ESTÁTUAS E OBJETOS
const renderM = (bufs, posArr, s, r = 0, isRoom = 0) => {
  if (!bufs) return;
  gl.uniform1i(uUseMTLColor, 1);
  gl.uniform1i(uIsRoomObject, isRoom);
  posArr.forEach(p => {
    let mm = multiplyMatrices(mat4Translate(p.x, p.y, p.z),
      multiplyMatrices(mat4RotateY(r), mat4Scale(s, s, s)));
    gl.uniformMatrix4fv(uModelviewMatrix, false, multiplyMatrices(viewMatrix, mm));
    desenharOBJComCores(gl, bufs, shaderProgram);
  });
};
```


Iluminação

Fragment shader

No fragment shader ocorre a etapa mais importante do pipeline gráfico no que diz respeito à aparência visual da cena, pois é nesse estágio que a iluminação, os efeitos de material e a simulação de superfícies especiais são calculados fragmento a fragmento. Esse tipo de abordagem permite um nível de realismo significativamente maior do que cálculos realizados apenas por vértice, já que cada pixel é tratado individualmente.

O primeiro passo do shader consiste na normalização dos vetores fundamentais utilizados no modelo de iluminação. A normal da superfície e a direção do observador são calculadas no espaço da câmera, o que simplifica os cálculos e torna a iluminação consistente independentemente da posição do objeto na cena.

```
vec3 N = normalize(vNormal);  
vec3 V = normalize(-vViewPos);  
vec3 L = V;
```

Iluminação

Fragment shader

A normal N define a orientação da superfície no fragmento atual, enquanto V representa a direção do observador em relação ao fragmento. Como a lanterna está acoplada à câmera, a direção da luz L coincide com a direção de visão, simulando uma fonte de luz portátil.

Em seguida, o shader implementa o modelo de lanterna por meio de um cone de luz suave (spotlight). A intensidade da iluminação depende do alinhamento do fragmento com a direção da câmera, criando um feixe central mais intenso e bordas suavizadas.

```
float theta = dot(normalize(vViewPos), cameraDir);  
float intensity = smoothstep(uCutoff - 0.12, uCutoff, theta);
```

Iluminação

Fragment shader

Esse controle evita transições bruscas de iluminação e reforça a sensação de foco da lanterna, contribuindo diretamente para a atmosfera da cena.

A iluminação principal segue o modelo de Phong, composto por termos difuso e especular. A componente difusa depende do ângulo entre a normal da superfície e a direção da luz, enquanto a componente especular simula reflexos sutis, ajustados para evitar o efeito visual de superfícies excessivamente brilhantes.

```
float diff = max(dot(N, L), 0.0);  
vec3 R = reflect(-L, N);  
float spec = pow(max(dot(R, V), 0.0), 8.0);
```


Iluminação

Fragment shader

A luz utilizada possui coloração levemente amarelada, evitando o aspecto artificial de luz branca pura e simulando melhor uma lanterna incandescente.

A definição da cor base do fragmento ocorre em seguida. Dependendo do tipo de objeto, essa cor pode ser obtida diretamente do material (cores provenientes do arquivo MTL ou dos vértices) ou calculada a partir de uma textura aplicada ao cenário.

```
if (uUseMTLColor == 1 || uIsKey == 1 || uIsRoomObject == 1) {  
    baseColor = vColor;  
}
```

Iluminação

Fragment shader

Para paredes e chão, o shader utiliza mapeamento triplanar, uma técnica que projeta a textura nos três eixos do espaço e combina os resultados de acordo com a inclinação da superfície. Essa abordagem elimina o esticamento de textura comum quando se utilizam coordenadas UV tradicionais.

```
vec3 xTex = texture(uStoneTexture, vWorldPos.zy * scale).rgb;  
vec3 yTex = texture(uStoneTexture, vWorldPos.xz * scale).rgb;  
vec3 zTex = texture(uStoneTexture, vWorldPos.xy * scale).rgb;
```

A contribuição de cada projeção é ponderada pela normal da superfície, garantindo transições suaves entre diferentes orientações geométricas.

```
baseColor = xTex * blending.x + yTex * blending.y + zTex * blending.z;
```

Iluminação

Fragment shader

Para regiões específicas do chão, o shader implementa um material especial que simula uma poça de sangue. Essa área é identificada por uma verificação espacial simples baseada na posição do fragmento e na orientação da superfície.

```
float distPoca = length(vWorldPos.xz - vec2(0.2, -0.3));  
if (N.y > 0.5 && distPoca < 0.4) {  
    baseColor = vec3(0.25, 0.0, 0.0);  
}
```

Iluminação

Fragment shader

Embora essa versão não utilize reflexão explícita por ray tracing, o efeito visual é obtido por meio de um ajuste cuidadoso da iluminação, da cor e do brilho especular, transmitindo a aparência de um material orgânico escuro e úmido sob a luz da lanterna.

Após a definição da cor base, o shader calcula a atenuação da luz em função da distância entre o fragmento e a câmera, simulando a perda natural de intensidade luminosa.

```
float attenuation = 1.0 / (1.0 + 0.25 * dist + 0.15 * (dist * dist));
```

Iluminação

Fragment shader

Uma componente de iluminação ambiente global é aplicada para garantir visibilidade mínima mesmo fora do feixe principal da lanterna, enquanto os termos difuso e especular são modulados pelo cone de luz e pela atenuação.

```
vec3 ambient = baseColor * ka;  
vec3 diffuse = baseColor * diff * lightColor;  
vec3 specular = vec3(1.0) * spec * ks;
```


Iluminação

Fragment shader

Por fim, todas as componentes de iluminação são combinadas para produzir a cor final do fragmento, resultando em uma cena com iluminação localizada, materiais bem definidos e forte sensação de profundidade e atmosfera.

```
fragColor = vec4(combinedLight, 1.0);
```

Matrizes

TCW

No início do pipeline, todos os objetos são definidos no Sistema de Coordenadas do Mundo (WCS). Nesse espaço, cada vértice possui posição, rotação e escala próprias, todas relativas a um referencial global fixo.

As transformações geométricas básicas — escala, rotação e translação — são representadas por matrizes 4×4.

```
function mat4Identity() {  
    return new Float32Array([  
        1, 0, 0, 0,  
        0, 1, 0, 0,  
        0, 0, 1, 0,  
        0, 0, 0, 1  
    ]);  
}
```

A matriz identidade representa um objeto sem qualquer transformação aplicada, servindo como ponto de partida para a composição das demais matrizes.

Matrizes

Translação e Escala

A translação desloca o objeto no espaço 3D, enquanto a escala altera seu tamanho ao longo dos eixos cartesianos.

```
function mat4Translate(x, y, z) {  
    return new Float32Array([  
        1, 0, 0, 0,  
        0, 1, 0, 0,  
        0, 0, 1, 0,  
        x, y, z, 1  
    ]);  
}  
  
function mat4Scale(sx, sy, sz) {  
    return new Float32Array([  
        sx, 0, 0, 0,  
        0, sy, 0, 0,  
        0, 0, sz, 0,  
        0, 0, 0, 1  
    ]);  
}
```

Essas matrizes são aplicadas ainda no WCS, definindo onde o objeto está e qual o seu tamanho no mundo.

Matrizes

Rotação

A orientação dos objetos é controlada pelas matrizes de rotação em torno dos eixos X (pitch), Y (yaw) e Z (roll).

```
function mat4RotateX(angle) {  
  const c = Math.cos(angle);  
  const s = Math.sin(angle);  
  return new Float32Array([  
    1, 0, 0, 0,  
    0, c, s, 0,  
    0, -s, c, 0,  
    0, 0, 0, 1  
  ]);  
}
```

```
function mat4RotateY(angle) {  
  const c = Math.cos(angle);  
  const s = Math.sin(angle);  
  return new Float32Array([  
    c, 0, -s, 0,  
    0, 1, 0, 0,  
    s, 0, c, 0,  
    0, 0, 0, 1  
  ]);  
}
```

```
function mat4RotateZ(angle) {  
  const c = Math.cos(angle);  
  const s = Math.sin(angle);  
  return new Float32Array([  
    c, s, 0, 0,  
    -s, c, 0, 0,  
    0, 0, 1, 0,  
    0, 0, 0, 1  
  ]);  
}
```

Essas matrizes definem a orientação espacial dos objetos antes de qualquer influência da câmera.

Matrizes

Transformação

As transformações não são aplicadas isoladamente. Elas são combinadas por multiplicação matricial, respeitando a ordem correta.

```
function multiplyMatrices(a, b) {  
    let out = new Float32Array(16);  
  
    for (let col = 0; col < 4; col++) {  
        for (let row = 0; row < 4; row++) {  
            out[col * 4 + row] =  
                a[0 * 4 + row] * b[col * 4 + 0] +  
                a[1 * 4 + row] * b[col * 4 + 1] +  
                a[2 * 4 + row] * b[col * 4 + 2] +  
                a[3 * 4 + row] * b[col * 4 + 3];  
        }  
    }  
    return out;  
}
```

O resultado dessa composição é a Model Matrix, que transforma vértices do espaço local do objeto para o WCS.

Matrizes

Visualização

Após posicionar os objetos no mundo, a cena precisa ser expressa no referencial da câmera. Isso é feito pela View Matrix, que converte coordenadas do WCS para o sistema da câmera.

```
function lookAt(eye, center, up) {  
    // cálculo dos eixos da câmera  
    ...  
    return new Float32Array([  
        x[0], y[0], z[0], 0,  
        x[1], y[1], z[1], 0,  
        x[2], y[2], z[2], 0,  
        -(x·eye), -(y·eye), -(z·eye), 1  
    ]);  
}
```

Essa matriz posiciona a câmera no mundo e define para onde ela está olhando, sem mover os objetos — apenas mudando o referencial.

Matrizes

Perspectiva

Depois da transformação de câmera, a cena ainda é tridimensional. A matriz de projeção é responsável por mapear esse espaço 3D para um volume de visualização.

```
function createPerspective(fovy, aspect, near, far) {  
    var fovyRad = fovy * Math.PI / 180.0;  
    var fy = 1 / Math.tan(fovyRad / 2.0);  
    var fx = fy / aspect;  
  
    return new Float32Array([  
        fx, 0, 0, 0,  
        0, fy, 0, 0,  
        0, 0, A, -1,  
        0, 0, B, 0  
    ]);  
}
```

A projeção perspectiva cria a ilusão de profundidade, fazendo objetos distantes parecerem menores.